
실무중심산학협력프로젝트1 3분반

프로젝트 최종 결과 보고서

팀 이름: MIML (Music Is My Life)

구성원: 소프트웨어학과 32232779 유소현, 32234852 한주연



목차

1. 프로젝트 개요 및 개발 목적
2. 요구사항 분석
3. 기존 서비스 분석
4. 시스템 설계
5. 개발 환경
6. 주요 기능 구현
7. 구현 결과
8. 기대 효과
9. 한계점 및 향후 개선 방향
10. 결론

1. 프로젝트 개요 및 개발 목적

1.1 프로젝트 개요

현대인의 음악 소비는 단순한 장르 선택을 넘어, 그 순간의 감정·상황·활동에 맞는 음악을 원하는 방향으로 진화하고 있다. 그러나 기존의 스트리밍 서비스는 단순 태그 검색이나 알고리즘 추천에 머물러 있어 사용자가 ‘지금 이 기분에 딱 맞는 곡’을 직관적으로 탐색하기 어렵다.

MIML(MusicIsMyLife)은 사용자가 자연어로 현재 감정·상황·활동을 입력하면, AI가 이를 분석하여 개인화된 음악 목록을 즉시 추천하는 Android 기반 음악 추천 앱이다.

항목	내용
프로젝트명	MusicIsMyLife (MIML)
개발 플랫폼	Android (React Native / Expo)
핵심 기술	OpenAI API, Spotify API, Firebase, Spring Boot
팀 구성	유소현 (백엔드/DB), 한주연 (프론트엔드)

1.2 개발 목적

MIML(MusicIsMyLife)은 이 문제를 해결하기 위해 자연어 기반 감성 인식과 하이브리드 추천 알고리즘을 결합한 안드로이드 음악 추천 앱이다. 사용자가 '오늘 한강 산책하면서 듣기 좋은 노래 추천해줘'와 같이 자연어로 입력하면, OpenAI API가 이를 감성 벡터로 변환하고, 시스템이 개인화된 음악 목록을 즉시 반환한다. 아래는 이번 프로젝트에서 설정한 주요 목표들이다.

자연어 기반 음악 추천(상황, 감정 이해)
원하는 시점에 즉시 믹스 생성
태그 + 오디오 특성 혼합 추천
추천 이유 자동 생성 기능
개인 취향 반영 강도 조절 기능

2. 요구사항 분석

2.1 사용자 시나리오

다음은 MIML의 핵심 사용 시나리오이다.

1. 앱 실행 → 로그인(Firebase Google 인증) → 홈 화면(자연어 입력 UI) 진입
2. "오늘 비 오는 날 집에서 공부할 때 들기 좋은 곡 추천해줘" 텍스트 입력
3. 시스템: OpenAI GPT-4o-mini → 감성 태그 [#비오는날, #공부할때, #차분한] + 오디오 목표값 (energy=25, happiness=40, danceability=20, acousticness=70, tempo=80) 추출. 장르 미연급 시 장르 필터 미적용
4. 시스템: 사용자 취향 프로필과 profileRatio 비율로 블렌딩 → blended 수치 산출
5. 시스템: audio_features 테이블에서 범위 필터링 + 태그 임베딩 의미 확장 매칭으로 후보 풀 구성 → 오디오 거리 점수 · 태그 커버리지 점수 합산 → 상위 N곡 반환
6. 슬라이더로 '현재 무드 : 누적 취향' 비율(profileRatio) 조절 (예: 7:3) → 실시간 추천 수치 재블렌딩
7. 결과 목록에서 곡 선택 → Spotify SDK 연동 재생
8. 재생 후 ♥ / ✕ 피드백 제출 → 학습률 $\alpha = 1/(n+10)$ 으로 users 테이블의 취향 프로필 즉시 업데이트
9. 커뮤니티 탭 → 게시물 작성 버튼 → 곡 검색 후 선택
10. 표준 태그 선택 (#비오는날, #공부할때, #차분한) + 짧은 감상평 작성 후 게시
11. 게시물 데이터 → Firestore 저장 / DB 미등록 곡이면 SoundNet 분석 후 audio_features 자동 저장 → music_tags 테이블 vote_count 업데이트

2.2 기능 요구사항

요구사항 ID	요구사항명	설명	우선순위
FR-AUTH-01	이메일 회원가입	이메일과 비밀번호로 계정을 생성할 수 있어야 한다.	상
FR-REC-01	자연어 입력 처리	현재 기분, 상황, 활동을 자연어로 입력할 수 있어야 한다.	상
FR-REC-02	OpenAI 자연어 파싱	자연어를 감성 태그 및 오디오 특성 목표값으로 변환해야 한다.	상
FR-REC-03	하이브리드 벡터 매칭	오디오 특성과 커뮤니티 태그를 결합한 통합 벡터로 추천을 수행해야 한다.	상
FR-REC-05	개인화 가중치 슬라이더	현재 의도와 누적 청취 기록의 반영 비율을 슬라이더로 조절할 수 있어야 한다.	중
FR-PLAY-01	음악 재생	추천된 음악을 앱 내에서 재생할 수 있어야 한다.	상
FR-PLAY-02	플레이리스트 관리	플레이리스트를 생성, 조회, 수정, 삭제할 수 있어야 한다.	중
FR-FB-01	추천 만족도 평가	추천 결과에 대해 만족/불만족 피드백을 제공할 수 있어야 한다.	상
FR-COM-01	감상 로그 작성	특정 곡을 지정하고 해시태그로 분위기를 정의하는 게시물을 작성할 수 있어야 한다.	중

3. 기존 서비스 분석

기존 음악 스트리밍 서비스의 한계를 분석하고, MIML의 차별점을 도출하였다.

▶ 기존 서비스의 한계

평가 항목	Spotify	YouTube Music	Melon	MIML (보완 방향)
자연어 문장 기반 의도 분석 검색	X	X	X	OpenAI API로 자연어 문장 의도 분석 후 추천
제한 없는 즉시 추천 생성	X	O	△	자연어 입력 즉시 추천 생성, 횟수 제한 없음
실시간 추천 가중치 직접 조절 (개인화)	X	X	△	슬라이더로 현재 의도 vs 과거 취향 비율 직접 조절
추천 근거 및 이유 제공 (투명성)	X	X	△	추천 이유 자동 생성, 가중치 반영 근거 표시
오디오 특성 벡터 기반 통합 추천	△	△	X	오디오 벡터 + 커뮤니티 태그 벡터 통합 임베딩
실시간 피드백 추천 반영 (즉각 업데이트)	X	X	X	좋아요·스킵·태그 피드백으로 가중치 실시간 업데이트
커뮤니티 데이터 추천 연동	△	X	X	유저 리뷰·태그가 추천 알고리즘 핵심 자료로 직접 사용
진입 장벽	X	X	X	캡스톤 프로토타입 — 무료 제공 목표

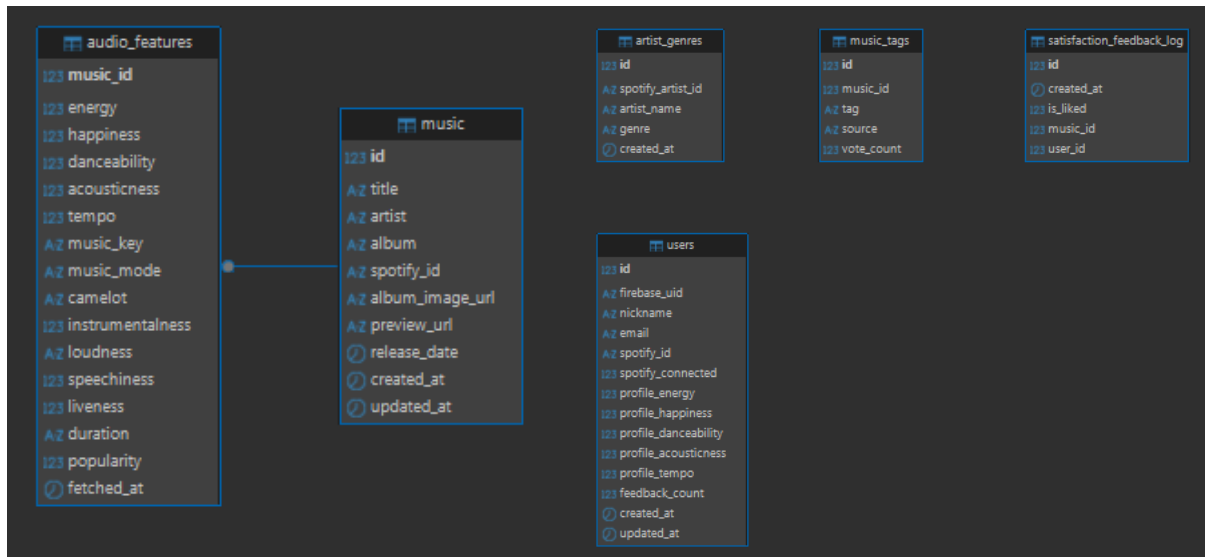
MIML은 자연어 의도 분석 기반 추천, 사용자 주도 가중치 조절, 커뮤니티 데이터의 알고리즘 직접 연동을 통해 기존 스트리밍 서비스의 불투명한 추천 방식을 탈피하고, 사용자 중심의 음악 추천 경험을 제공한다."

4. 시스템 설계

4.1 시스템 아키텍처

레이어	구성 요소 및 역할
Client Layer	Android App (Kotlin) — 사용자 인터페이스, 자연어 입력, 추천 결과 표시, 재생
API Gateway	Spring Boot REST API (EC2) — 인증, 라우팅, NLP 연동, 추천 로직 실행
AI/NLP Layer	OpenAI API — 자연어 파싱, 감성 태그 및 오디오 특성 벡터 생성
Data Layer	Amazon RDS MySQL — 사용자, 음악, 피드백, 태그, 취향 벡터 저장
Community Layer	Firebase Firestore — 실시간 커뮤니티 게시물 읽기/쓰기
External Data	Music API — 음악 메타데이터 및 오디오 특성 초기 수집 및 저장

4.2 DB 구조(ERD)



4.3 핵심 테이블 정의

테이블명	저장 데이터 및 역할역할
audio_features	SoundNet으로 수집한 곡별 오디오 수치 (energy, happiness, danceability, acousticness, tempo, popularity 등 16개 컬럼). 추천 점수 계산의 핵심 데이터.
music	곡 메타데이터 (title, artist, album, spotify_id, album_image_url, preview_url 등). 추천 결과 화면 표시에 사용.
music_tags	커뮤니티가 곡에 부여한 감성 해시태그(tag)와 투표 수(vote_count), 출처(source). 태그 커버리지 점수 계산에 활용.
artist_genres	Last.fm에서 수집한 아티스트별 장르 태그. 장르 include/exclude 필터 적용 시 아티스트 조회에 사용.
users	Firebase UID 기반 사용자 정보 및 개인화 취향 프로파일 (profile_energy, profile_happiness, profile_danceability, profile_acousticness, profile_tempo, feedback_count). 블렌딩 및 피드백 학습에 사용.

4.4 추천

알고리즘 설계

음악 추천 알고리즘은 아래 5단계 파이프라인으로 설계하였다.

① 자연어 → 오디오 수치 변환 (GPT-4o-mini) : 사용자의 자연어 입력을 5가지 오디오 피쳐 수치와 감성 태그로 변환한다.

② 개인화 블렌딩 : 변환된 무드 수치와 사용자의 누적 취향 프로필을 비율로 혼합한다.

계산식: $\text{blended} = (1 - \text{profileRatio}) \times \text{moodValue} + \text{profileRatio} \times \text{profileValue}$

③ 후보 풀 추출 : 블렌딩 수치 기준 $\pm 35\text{pt}$ 범위 필터링 후, text-embedding-3-small 기반 코사인 유사도(≥ 0.75)로 감성 태그를 의미 확장하여 후보 곡을 선별한다. 장르 명시 시 장르 필터 추가 적용한다.

④ 2단계 점수화 및 순위 결정

계산식: $\text{audioScore} = 1 - (\text{distance} / 80)$ (5개 피쳐 유클리드 거리) $\text{tagScore} = \text{avg}(\max \text{cosine similarity}) \times (1 + \text{voteBonus})$ 최종 점수 = $\text{tagScore} \times 0.85 + \text{popularityScore} \times 0.15$

⑤ 다양성 보장 샘플링 : $\text{score}^{0.5}$ 가중 샘플링 + 아티스트당 최대 1곡 제한으로 최종 N곡을 선별한다.

⑥ 피드백 학습 : 좋아요/싫어요 피드백을 적응형 학습률 $\alpha = 1/(n+10)$ 으로 취향 프로필에 반영하여 이용할수록 추천 정확도가 향상된다.

5. 개발 환경

5.1 프론트엔드

항목	내용
프레임워크	React Native (Expo SDK 54)
언어	JavaScript (ES2022)
상태 관리	React Context API (PlayerContext, AuthContext, SpotifyContext)
네비게이션	React Navigation (Bottom Tab + Stack Navigator)
인증	Firebase Authentication
DB	Firebase Firestore (커뮤니티 게시물)
음악 재생	Spotify SDK 연동
IDE	VS Code
버전 관리	Git / GitHub

5.2 백엔드

항목	내용
프레임워크	Spring Boot 3.x
언어	Kotlin
빌드 도구	Gradle (Kotlin DSL)
DB	MYSQL 8.0 (AWS RDS)
ORM	Spring Data JPA/Hibernate
인증	Firebase Authentication (Admin SDK)
배포	AWS EC2 (Amazon Linux 2023, Java 17)
IDE	IntelliJ IDEA

5.3 외부 API

API / 라이브러리	용도
OpenAI API	자연어 파싱 → 감성 태그 및 오디오 특성 벡터 생성
Spotify API	곡 정보 수집, 앨범 이미지, 음악 재생 제어
Last.fm	곡 정보 보조 수집
Firebase SDK	실시간 커뮤니티 게시물 읽기/쓰기, 로그인 기능

6. 주요 기능 구현

6.1 자연어 기반 음악 추천 (프론트엔드)

홈 화면은 채팅 UI 형태로 구성되어, 사용자가 자연어로 상황을 입력하면 AI가 응답하는 방식으로 동작한다.

- 사용자 입력 → 백엔드 `/api/recommend/smart` 호출
- 추천 결과를 채팅 버블 형태로 표시 (곡명, 아티스트, 앨범 이미지)
- 각 곡에 좋아요/싫어요 피드백 버튼 제공
- '전체 재생' 버튼으로 추천 목록 큐에 추가하여 순서대로 재생
- '플레이리스트로 저장' 버튼으로 현재 추천 목록을 보관함에 저장

6.2 개인화 슬라이더

화면 상단에 'Now ↔ My Vibe' 슬라이더를 배치하여, 사용자가 α 값을 실시간으로 조절할 수 있도록 구현하였다. 슬라이더는 PanResponder를 이용한 커스텀 컴포넌트로 제작되었다.

6.3 Spotify 연동 재생

NowPlayingBar 컴포넌트에서 Spotify SDK를 통해 음악 재생을 제어한다. 새 곡이 선택될 때만 재생이 시작되도록 isMounted 가드를 적용하여, 탭 전환 시 노래가 재시작되는 버그를 수정하였다.

6.4 보관함 (Library)

- Liked Songs: 좋아요 표시한 곡 자동 저장
- Recently Played: 최근 재생 목록 자동 기록 (최대 20 곡)
- 사용자 생성 플레이리스트: 직접 이름 지정하여 생성 및 관리
- 플레이리스트 데이터는 Firebase Firestore 에 동기화

6.5 커뮤니티

사용자는 특정 곡을 선택하고 감상 태그와 짧은 감상평을 작성하여 커뮤니티에 게시할 수 있다. 게시물은 Firebase Firestore에 저장되며, 백엔드 DB의 music_tag_map 가중치 업데이트에 활용된다.

6.6 백엔드 핵심 로직

사용자 요청 시 아래 5단계가 순차적으로 실행된다.

STEP 1	자연어 분석 (GPT-4o-mini) — 입력 문장 → 오디오 피쳐 수치 5종 + 감성 태그 2~4개 + 장르 include/exclude 추출
STEP 2	유저 프로필 블렌딩 — 무드 수치와 개인 취향 프로필을 profileRatio 비율로 선형 혼합
STEP 3	후보 풀 추출 — 오디오 범위 필터($\pm 35\text{pt}$) + 태그 의미 확장 매칭 + 장르 필터로 후보 곡 선별
STEP 4	2단계 점수화 — ① 오디오 거리 점수로 상위 풀 확정, ② 태그 커버리지 점수로 최종 순위 결정
STEP 5	가중 샘플링 + 아티스트 중복 제거 — $\text{score}^{0.5}$ Temperature Scaling + 아티스트당 최대 1곡 제한으로 최종 N곡 선별

Step 1. 자연어 분석 (GPT-4o-mini)

사용자 입력을 GPT-4o-mini에 전달하여 오디오 피쳐 5종(energy, happiness, danceability, acousticness, tempo), 감성 태그 2~4개, 장르 필터(genreInclude/genreExclude)를 JSON으로 추출한다. GPT가 장르를 놓치는 경우를 대비해 키워드 사전 기반 보조 감지(detectGenreFromInput)를 병렬 수행하여 결과를 합산한다.

- happiness 측은 에너지가 아닌 감정의 긍정성 측정 (분노/스트레스 → 0~15)
- 허용 태그는 TagVocabulary에 사전 정의된 약 80 개로만 제한
- 장르 키워드("케이팝", "힙합" 등)는 NFC 정규화 후 canonical key로 매핑

Step 2. 유저 프로필 블렌딩

GPT 분석 수치와 사용자 취향 프로필을 profileRatio 비율로 선형 혼합한다.

$$\text{blendedValue} = (1 - \text{profileRatio}) \times \text{moodValue} + \text{profileRatio} \times \text{profileValue}$$

profileRatio=0이면 입력 무드 100% 반영, profileRatio=1이면 개인 프로필 100% 반영. 태그 장르 필터는 블렌딩 없이 항상 GPT 분석 결과만 사용한다.

Step 3. 후보 풀 추출

- 오디오 범위 필터: 블렌딩 수치 기준 $\pm 35\text{pt}$ 범위(tempo는 $\pm 70\text{ BPM}$) 내 곡을 DB에서 조회
- 태그 의미 확장: GPT 태그와 코사인 유사도 ≥ 0.75 인 TagVocabulary 태그까지 확장하여 추가 후보 선별 (text-embedding-3-small 사용)
- 두 풀을 합산 후 장르 명시 시 artist_genres 기반 include/exclude 필터 적용

Step 4. 2단계 점수화

① 오디오 거리 점수 (Audio Score)

5개 피처의 유클리드 거리를 계산. tempo는 $(\text{tempo} - 60) / 1.4$ 로 0~100 스케일 정규화 후 적용.

$$\text{distance} = \sqrt{(\sum (\text{feature}_i - \text{target}_i)^2 / 5)} \quad \text{audioScore} = 1 - (\text{distance} / 80)$$

audioScore ≥ 0.45 인 곡만 통과, 상위 N×6곡을 오디오 풀로 확정.

② 태그 커버리지 점수 (Tag Score)

GPT 생성 태그 각각이 커뮤니티 태그에서 얼마나 커버되는지 임베딩 코사인 유사도로 평균. voteCount 기반 최대 +20% 보정.

$$\text{coverageScore} = \text{avg}(\max \cos(\text{genTag}_i, \text{commTag}))$$
$$\text{tagScore} = \text{coverageScore} \times (1 + \ln(\text{voteCount})/\ln(200)) \quad [\text{clamp } 0 \sim 1]$$

③ 최종 점수 합산

상황	최종 점수
커뮤니티 태그 존재	$\text{tagScore} \times 0.85 + \text{popularityScore} \times 0.15$
커뮤니티 태그 없음	$\text{audioScore} \times 0.85 + \text{popularityScore} \times 0.15$

정렬 후 ± 0.08 랜덤 노이즈(Diversity Jitter)를 추가하여 동점권 곡의 순서를 매 요청마다 다르게 하여 다양성을 확보한다.

STEP5. 가중 샘플링 + 아티스트 중복 제거

점수를 가중치로 사용하는 확률적 샘플링. SCORE_TEMPERATURE=0.5 적용으로 $\text{score}^{0.5}$ 를 가중치로 사용하여 점수 격차를 압축, 하위권 곡도 선택될 기회를 부여한다. 아티스트당 최대 1곡(MAX_PER_ARTIST=1)으로 특정 아티스트 독점을 방지한다.

추가적으로, 좋아요/싫어요 피드백으로 사용자의 오디오 피쳐 프로필(profile_energy ~ profile_tempo)을 점진적으로 업데이트한다.

$$\alpha = 1 / (\text{feedbackCount} + 10)$$

$$\text{좋아요: newProfile} = \text{profile} + \alpha \times 1.0 \times (\text{songFeature} - \text{profile})$$

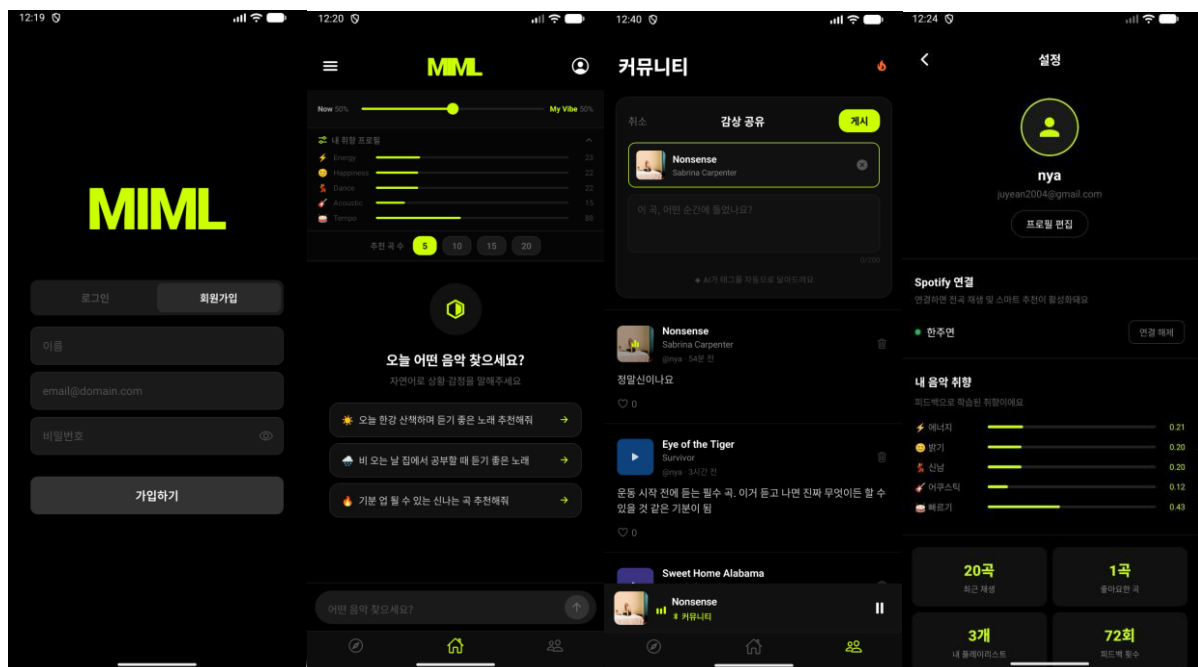
$$\text{싫어요: newProfile} = \text{profile} + \alpha \times (-0.5) \times (\text{songFeature} - \text{profile})$$

학습률 α 는 초기(feedbackCount=0)에 0.1로 빠르게 학습하다가, 피드백이 쌓일수록 점차 낮아져 프로필이 안정화된다. 싫어요는 절반 강도(-0.5)를 적용하여 급격한 프로필 변동을 방지한다. 동일 곡에 대한 중복 피드백은 차단된다.

7. 구현 결과

7.1 프론트엔드 화면 구현

화면	구현 내용
로그인 화면	MIML 로고, 이메일/비밀번호 입력, Google 소셜 로그인
홈 (추천/채팅)	채팅 버블 형태의 자연어 입력 UI, 개인화 슬라이더, 추천 곡 목록, 하단 미니 플레이어
보관함	Recently Played, Liked Songs, 사용자 생성 플레이리스트 목록
커뮤니티	감상 로그 게시물 목록, 작성 버튼, 게시물 상세
재생 화면	현재 재생 곡 정보, 재생/일시정지 제어, 다음/이전 곡
프로필	사용자 정보 조회, 취향 프로필 시각화



7.2 백엔드 구현 결과

```
24 POST {{baseUrl}}/api/recommend/smart
25 Content-Type: application/json
26 Authorization: Bearer {{firebaseToken}}
27
28 {
29   "description": "새벽에 혼자 드라이브하며 감성에 젖고 싶어",
30   "profileRatio": 0.0,
31   "limit": 10
32 }
```

POST http://localhost:8080/api/recommend/smart

요청 표시

HTTP/1.1 200

> (헤더) ...Content-Type: application/json...

```
{
  "totalCandidates": 186,
  "returnedCount": 10,
  "recommendations": [
    {
      "musicId": 1430,
      "title": "\"Life on Pluto\" in Ab Major, Op. 2: \"I. Neo-Romanticism\"",
      "artist": "Jaylen Spencer",
      "album": "LIFE ON PLUTO DELUXE",
      "albumImageUrl": "https://i.scdn.co/image/ab67616d0000b273b916b70315ff9a51f39d7bb6",
      "spotifyId": "6xWSmm47iEWmov6Zv1CEKj",
      "score": 0.3055782633012266
    },
    {
      "musicId": 222,
      "title": "Happier Than Ever",
      "artist": "Billie Eilish",
      "album": "Halloween",
      "albumImageUrl": "https://i.scdn.co/image/ab67616d0000b2732a038d3bf875d23e4aeaa84e",
      "spotifyId": "4RVwu0a32PAaUiJoXsdF8"
```

/api/recommend/smart 엔드포인트에 자연어 입력("새벽에 혼자 드라이브하며 감성에 젖고 싶어")을 담아 POST 요청을 전송한 결과이다. HTTP 200 응답과 함께 총 186개의 후보 곡 중 최종 10곡이 반환되었으며, 각 곡의 제목·아티스트·앨범·앨범 이미지 URL·Spotify ID·추천 점수(score)가 포함된 JSON 형식으로 응답됨을 확인할 수 있다

8. 기대 효과

- 기존 키워드 기반 추천의 한계 개선: 자연어로 '한강 산책하면서 들을 노래'처럼 직관적으로 음악을 탐색할 수 있다.
- 사용자 상황 중심 음악 탐색 경험 제공: 특정 장르나 아티스트를 몰라도 상황과 감정만으로 원하는 음악을 찾을 수 있다.
- 사용할수록 정교해지는 개인화: 피드백 루프를 통해 추천 알고리즘이 점진적으로 개인 취향에 최적화된다.
- 커뮤니티 기반 감성 데이터셋 구축: 사용자 감상 로그가 쌓일수록 알고리즘의 감성-음악 매핑 품질이 향상된다.
- 음악 발견의 새로운 경험: 평소 접하지 못했던 곡들을 상황 맥락 기반으로 자연스럽게 발견할 수 있다.

10. 한계점 및 향후 개선 방향

10.1 현재 한계점

- 음악 DB 규모: 초기 수집된 음악 데이터 수가 제한적이어서 추천 다양성에 한계가 있다.
- Spotify 프리미엄 의존: Spotify 재생 제어 기능은 프리미엄 계정이 필요하다.
- 콜드 스타트 문제: 신규 사용자는 `profile_vector` 가 없어 초기 개인화 추천 품질이 낮다.
- 커뮤니티 데이터 부족: 서비스 초기에는 커뮤니티 태그 데이터가 부족하여 태그 기반 추천 정확도가 낮다.
- iOS 미지원: 현재 Android 에뮬레이터 기준으로만 구현 및 테스트되었다.

10.2 향후 개선 방향

- 음악 DB 확장: 더 많은 음악 데이터를 수집하여 추천 다양성을 높인다.
- 음성 입력 지원: 마이크 아이콘을 통한 음성 입력 기능을 완성한다.
- iOS 지원: React Native 기반이므로 iOS 빌드를 추가로 지원한다.
- 추천 설명 강화: 각 곡이 추천된 구체적인 이유(오디오 특성 수치)를 사용자에게 시각적으로 표시한다.
- 소셜 기능 강화: 커뮤니티 내 다른 사용자 팔로우 및 플레이리스트 공유 기능을 추가한다.

11. 결론

MIML(Music Is My Life)은 "지금 이 순간의 감정에 맞는 음악"을 찾기 어렵다는 문제의식에서 출발하였다. 장르나 아티스트를 몰라도 현재 상황을 자연어로 설명하기만 하면 AI가 이를 해석하여 어울리는 곡을 즉시 제안하는 Android 음악 추천 앱으로, GPT-4o-mini 기반 자연어 파싱, 오디오 특성 벡터 매칭, 커뮤니티 태그 임베딩을 결합한 하이브리드 추천 파이프라인을 핵심으로 설계하였다. 프론트엔드는 React Native(Expo), 백엔드는 Spring Boot와 MySQL로 구성하였으며 OpenAI, Spotify, Firebase 등 다수의 외부 서비스를 통합하였다.

개발 과정에서 Google 소셜 로그인, 자연어 입력 기반 추천, profileRatio 슬라이더를 통한 실시간 블렌딩 조절, Spotify SDK 재생, 좋아요·싫어요 피드백에 따른 취향 프로필 자동 업데이트, 보관함 및 플레이리스트 관리, 커뮤니티 감상 로그 기능을 구현하고 Android 환경에서 정상 동작을 확인하였다. 특히 사용자가 커뮤니티에 남긴 감성 태그가 추천 점수 산정에 직접 활용되도록 설계하여, 이용자가 늘수록 알고리즘의 감성-음악 매핑 정밀도가 함께 높아지는 구조를 갖추었다.

현재는 초기 수집된 음악 DB 규모와 Android 단일 플랫폼이라는 제약이 있으나, 향후 데이터 확장, iOS 지원, 음성 입력 도입, 커뮤니티 소셜 기능 추가를 통해 완성도를 높여갈 계획이다.

이번 프로젝트를 통해 자연어 처리 API 활용, 추천 알고리즘 설계, 멀티 플랫폼 외부 API 통합, 실시간 데이터 처리 등 강의실에서 접하기 어려운 기술 스택을 직접 다루는 경험을 쌓을 수 있었다. 무엇보다 사용자의 감정을 음악으로 연결한다는 아이디어를 실제로 동작하는 서비스로 구현해냈다는 점에서 본 프로젝트의 가장 큰 의미를 찾을 수 있다.